Editor.js - Internal Flow Diagrams

📋 Table of Contents

---

1. Initialization Flow

```
┌─────────────────────────────────────────────────────────────┐
│                    EDITOR INITIALIZATION                    │
└─────────────────────────────────────────────────────────────┘

[1] new Editor(eventBus, stateManager, container)
     │
     ├─> Store dependencies
     │    ├─> this.eventBus = eventBus
     │    ├─> this.stateManager = stateManager
     │    └─> this.container = container (DOM element)
     │
     ├─> Initialize viewport state
     │    ├─> { x: 0, y: 0, zoom: 1 }
     │    └─> { minZoom: 0.1, maxZoom: 5 }
     │
     ├─> Initialize grid settings
     │    └─> { enabled: true, size: 20, visible: true }
     │
     └─> Initialize interaction state
          └─> { isPanning: false, panStart: {x, y} }

[2] editor.initialize()
     │
     ├─> [A] createSVGStructure()
     │    │
     │    ├─> Create main <svg> element
     │    │    └─> Set width="100%", height="100%"
     │    │
     │    ├─> Create <defs> layer
     │    │    └─> Add arrow markers, patterns
     │    │
     │    ├─> Create viewport <g> group
     │    │    └─> For zoom/pan transformations
     │    │
```

```
        │    ├─> Create grid layer <g>
        │    │   └─> For grid lines
        │    │
        │    ├─> Create edge layer <g>
        │    │   └─> For connections (below nodes)
        │    │
        │    ├─> Create node layer <g>
        │    │   └─> For shapes
        │    │
        │    ├─> Create overlay layer <g>
        │    │   └─> For selection boxes, guides
        │    │
        │    └─> Append SVG to container
        │
        ├─> [B] setupEventHandlers()
        │    │
        │    ├─> Wheel event → handleWheel() (zoom)
        │    ├─> Mouse events → handleMouseDown/Move/Up() (pan)
        │    ├─> Click event → handleClick()
        │    ├─> Context menu → handleContextMenu()
        │    └─> Drag events → handleDragOver/Drop()
        │
        ├─> [C] renderGrid()
        │    └─> Draw grid pattern in grid layer
        │
        └─> [D] updateTransform()
            └─> Apply initial viewport transform

  ✓ Editor Ready!
```

---

2. Node Rendering Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                      NODE RENDERING FLOW                         │
└──────────────────────────────────────────────────────────────────┘

[App] Calls: editor.renderNode(nodeId, nodeData, shapeDefinition)
                        ↓
┌──────────────────────────────────────────────────────────────────┐
│ renderNode(nodeId, nodeData, shapeDefinition)                    │
└──────────────────────────────────────────────────────────────────┘
                        │
                        │
      ┌─────────────────┴──────────────────┐
      │                                    │
      ▼                                    ▼
[STEP 1: Create Node Container]   [STEP 2: Get Shape Definition]
      │                                    │
      ├─> Create <g> element               ├─> shapeDefinition ||
      ├─> Set data-node-id="nodeId"        │   getShapeDefinition(type)
      ├─> Set class="node"                 │
```

```
     └─> Set transform="translate(x,y)"  └─> Returns shape config
                        │
                        ▼
              [STEP 3: Create Shape Element]
                        │
          ┌─────────────┴──────────────┐
          │                            │
          ▼                            ▼
createShapeElement(shape, nodeData)    │
          │                            │
          ├─> Switch on shape.type     │
          │   ├─> 'rect' → <rect>      │
          │   ├─> 'circle' → <circle>  │
          │   ├─> 'diamond' → <path>   │
          │   └─> 'ellipse' → <ellipse>
          │                            │
          ├─> Apply styles             │
          │   ├─> fill                 │
          │   ├─> stroke               │
          │   └─> stroke-width         │
          │                            │
          └─> Set class="shape-element"
                        │
                        ▼
              [STEP 4: Create Label]
                        │
          createLabelElement(nodeData, shape)
                        │
          ├─> Create <text> element
          ├─> Position at center (width/2, height/2)
          ├─> Set text-anchor="middle"
          ├─> Set content = nodeData.label
          └─> Set class="node-label"
                        │
                        ▼
              [STEP 5: Create Ports]
                        │
          createPortsGroup(nodeId, nodeData, shape)
                        │
          ├─> Create <g class="ports">
          │
          ├─> For each port position (top, right, bottom, left):
          │   │
          │   ├─> Create <circle> port
          │   ├─> Set data-port-id="top|right|bottom|left"
          │   ├─> Set data-node-id="nodeId" ⭐
          │   ├─> Set fill="#2196F3"
          │   ├─> Set r="6"
          │   └─> Add mousedown handler → handlePortMouseDown(e, nodeId,
port)
          │
          └─> Set display="none" (hidden by default, shown on hover)
                        │
                        ▼
```

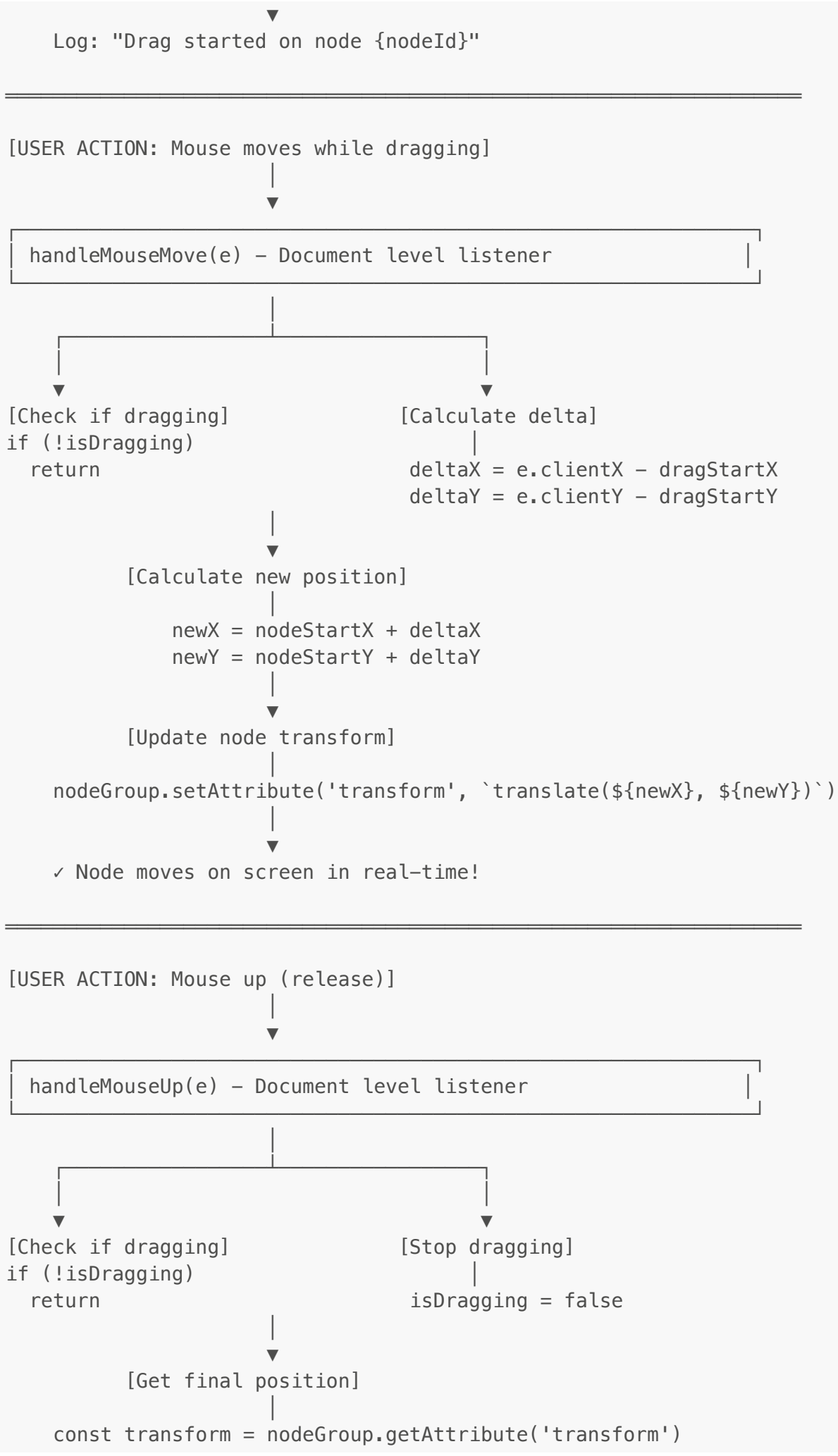
```
            [STEP 6: Create Resize Handles]
                        │
          createResizeHandles(nodeId, nodeData)
                        │
          ├─> Create <g class="resize-handles">
          │
          ├─> For each handle (nw, n, ne, e, se, s, sw, w):
          │   │
          │   ├─> Create <rect> handle
          │   ├─> Set data-handle-id="nw|n|ne|..."
          │   ├─> Set data-node-id="nodeId" ⭐
          │   ├─> Position at corner/side
          │   ├─> Set cursor style (nw-resize, n-resize, etc.)
          │   └─> Add mousedown handler → handleResizeStart(e, nodeId,
  handle)
          │
          └─> Set display="none" (hidden, shown on selection)
                        │
                        ▼
            [STEP 7: Setup Interactions]
                        │
          setupNodeInteractions(nodeGroup, nodeId, nodeData)
                        │
          ├─> Setup Drag Handlers (NEW!)
          │   ├─> mousedown → Start drag
          │   ├─> mousemove → Update position
          │   └─> mouseup → Finish drag, emit 'node:moved'
          │
          ├─> Hover → Show/hide ports
          ├─> Click → Select node, show resize handles
          └─> Double-click → Emit 'node:edit'
                        │
                        ▼
            [STEP 8: Add to Canvas]
                        │
          addNodeElement(nodeId, nodeGroup)
                        │
          └─> this.nodeLayer.appendChild(nodeGroup)
                        │
                        ▼
              ✓ Node Rendered on Canvas!

  ┌──────────────────────────────────────────────────────────────────┐
  │                    FINAL SVG STRUCTURE                           │
  ├──────────────────────────────────────────────────────────────────┤
  │                                                                  │
  │  <g class="node" data-node-id="node_123" transform="...">        │
  │    │                                                            │
  │    ├─> <rect class="shape-element">      ← The visible shape     │
  │    │                                                            │
  │    ├─> <text class="node-label">         ← The label text        │
  │    │                                                            │
  │    ├─> <g class="ports-group">           ← Connection ports      │
  │    │   ├─> <circle data-port-id="top">                          │
```

```
│    │   ├─> <circle data-port-id="right">                          │
│    │   ├─> <circle data-port-id="bottom">                         │
│    │   └─> <circle data-port-id="left">                           │
│    │                                                              │
│    └─> <g class="handles-group">         ← Resize handles        │
│        ├─> <rect data-handle-id="nw">                             │
│        ├─> <rect data-handle-id="n">                              │
│        ├─> <rect data-handle-id="ne">                             │
│        ├─> <rect data-handle-id="e">                              │
│        ├─> <rect data-handle-id="se">                             │
│        ├─> <rect data-handle-id="s">                              │
│        ├─> <rect data-handle-id="sw">                             │
│        └─> <rect data-handle-id="w">                              │
│  </g>                                                             │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

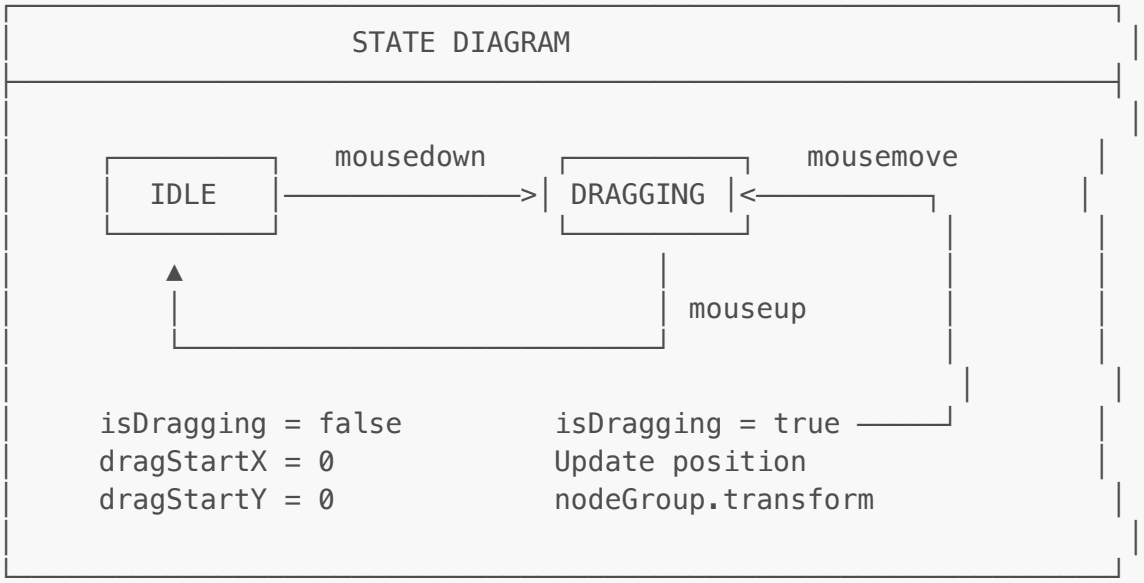
---

3. Node Dragging Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                      NODE DRAG FLOW                              │
└──────────────────────────────────────────────────────────────────┘

[USER ACTION: Mouse down on node shape]
                      │
                      ▼
┌──────────────────────────────────────────────────────────────────┐
│ handleMouseDown(e) - Inside setupNodeInteractions()              │
└──────────────────────────────────────────────────────────────────┘
                      │
                      ├──────────────────────────┐
                      │                          │
                      ▼                          ▼
[Check button]                          [Set drag state]
if (e.button !== 0)                           │
  return                                isDragging = true
                                        dragStartX = e.clientX
                                        dragStartY = e.clientY
                      │
                      ▼
          [Extract current position]
                      │
    const transform = nodeGroup.getAttribute('transform')
    const match = transform.match(/translate\(([^,]+),\s*([^)]+)\)/)
    nodeStartX = parseFloat(match[1])  // Current X
    nodeStartY = parseFloat(match[2])  // Current Y
                      │
                      ▼
            e.stopPropagation()
                      │
```

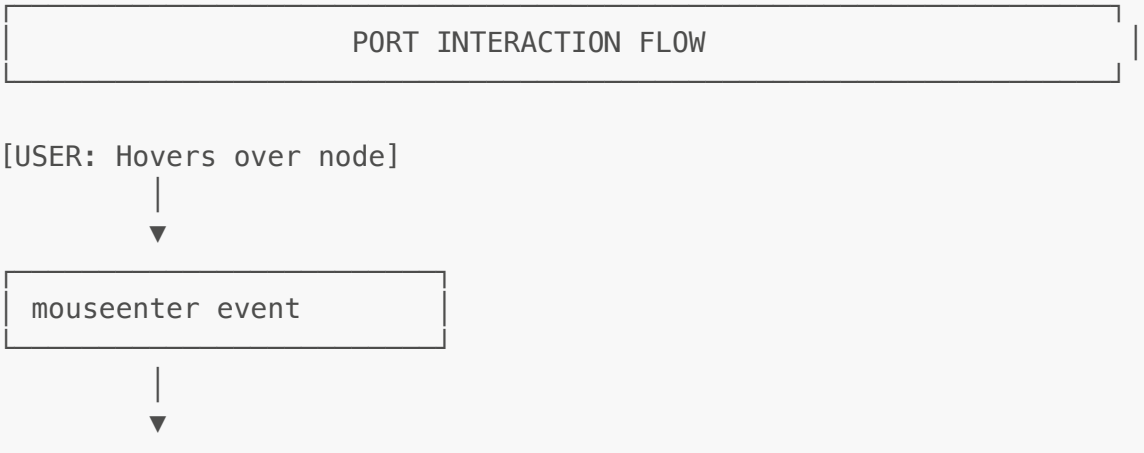

Log: "Drag started on node {nodeId}"
[USER ACTION: Mouse moves while dragging]
handleMouseMove(e) - Document level listener
[Check if dragging]
if (!isDragging)
return
[Calculate delta]
deltaX = e.clientX - dragStartX
deltaY = e.clientY - dragStartY
[Calculate new position]
newX = nodeStartX + deltaX
newY = nodeStartY + deltaY
[Update node transform]
nodeGroup.setAttribute('transform', `translate(${newX}, ${newY})`)
✓ Node moves on screen in real-time!
[USER ACTION: Mouse up (release)]
handleMouseUp(e) - Document level listener
[Check if dragging]
if (!isDragging)
return
[Stop dragging]
isDragging = false
[Get final position]
const transform = nodeGroup.getAttribute('transform')

```
    const match = transform.match(/translate\(([^,]+),\s*([^)]+)\)/)
    finalX = parseFloat(match[1])
    finalY = parseFloat(match[2])
                      │
                      ▼
          [Emit event with final position]
                      │
    eventBus.emit('node:moved', {
      nodeId: nodeId,
      x: finalX,
      y: finalY
    })
                      │
                      ▼
    Log: "Node {nodeId} moved to ({finalX}, {finalY})"
                      │
                      ▼
    ✓ Drag complete! Other components can react to 'node:moved' event
```



---

4. Port Interaction Flow



```
const portsGroup = nodeGroup.querySelector('.ports-group')
portsGroup.style.display = 'block'
         │
         ▼
✓ Ports become visible (4 blue circles at top, right, bottom, left)

═══════════════════════════════════════════════════════════════════

[USER: Clicks on a port (e.g., right port)]
         │
         ▼
┌──────────────────────────────────────────────────────────────────┐
│ Port mousedown event → handlePortMouseDown(e, nodeId, port)      │
└──────────────────────────────────────────────────────────────────┘
         │
         ├─> e.stopPropagation()  (Don't trigger node drag)
         │
         ├─> Log: "Starting edge from node {nodeId}, port {port.id}"
         │
         └─> Emit event:
             │
             eventBus.emit('port:mousedown', {
               nodeId: nodeId,           ← ✓ Now correct! (was undefined)
               portId: port.id,          ← 'top', 'right', 'bottom',
'left'
               portPosition: port.position,
               clientX: e.clientX,
               clientY: e.clientY
             })
         │
         ▼
┌──────────────────────────────────────────────────────────────────┐
│ [app.js] Receives 'port:mousedown' event                         │
└──────────────────────────────────────────────────────────────────┘
         │
         ├─> this.stateManager.setViewportMode("connecting")
         │
         └─> Store connection info:
             this.connectingFrom = {
               nodeId: data.nodeId,
               portId: data.portId,
               portPosition: data.portPosition
             }
         │
         ▼
┌──────────────────────────────────────────────────────────────────┐
│ [EdgeController] Can handle edge drawing                         │
└──────────────────────────────────────────────────────────────────┘
         │
         └─> Show temporary line from source port to mouse cursor
         │
         ▼
    [USER: Clicks on another port]
         │
```

```
          ▼
    Create edge connection between nodes!

┌──────────────────────────────────────────────────────────────────────┐
│                    PORT DATA FLOW                                    │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  createPortsGroup(nodeId, nodeData, shape)                           │
│         │                                                            │
│         └─> Create 4 ports with data:                                │
│                                                                      │
│     Top Port:                                                        │
│      ┌──────────────────────────────────────────────┐                │
│      │ <circle                                      │                │
│      │   data-port-id="top"                         │                │
│      │   data-node-id="node_123"      ← Added!      │                │
│      │   cx={width/2}                               │                │
│      │   cy={0}                                     │                │
│      │   r="6"                                      │                │
│      │   fill="#2196F3"                             │                │
│      │   cursor="crosshair"                         │                │
│      │   onclick="handlePortMouseDown(e, nodeId)"   │                │
│      │ />                                           │                │
│      └──────────────────────────────────────────────┘                │
│                                                                      │
│     Similarly for: right, bottom, left ports                         │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

5. Resize Handle Flow

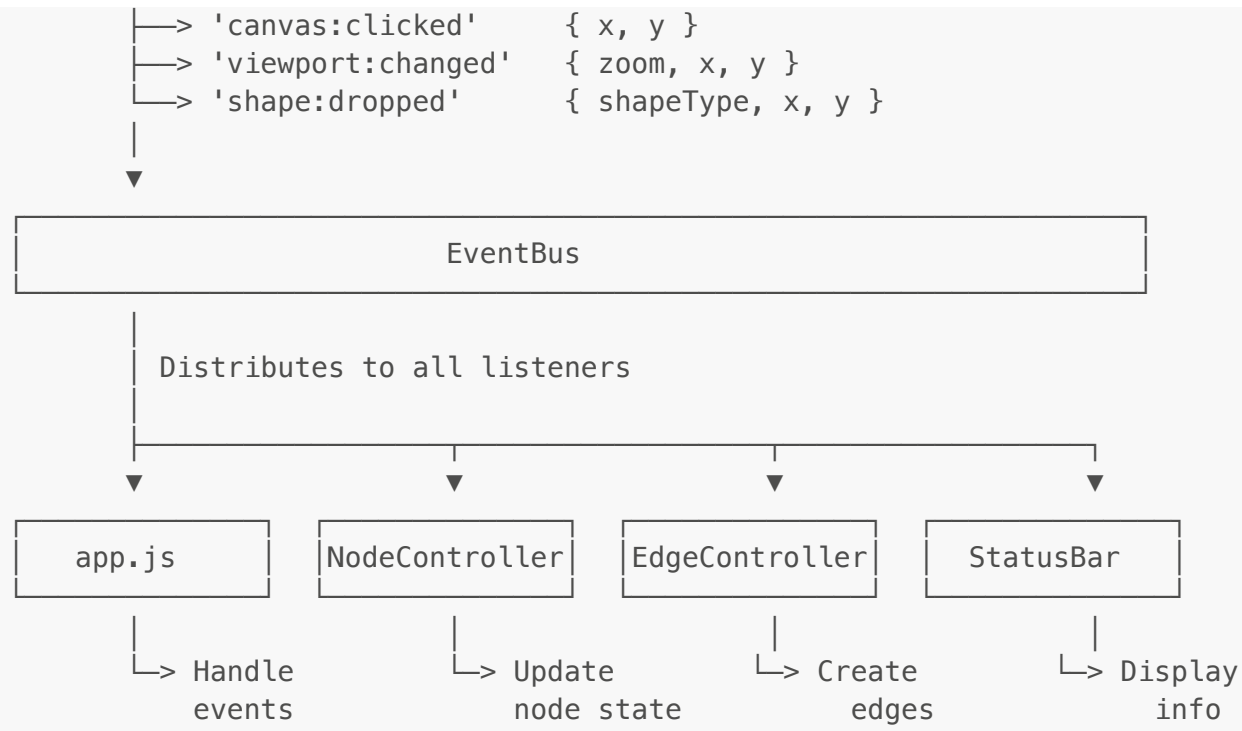
```
┌──────────────────────────────────────────────────────────────────────┐
│                  RESIZE HANDLE FLOW                                  │
└──────────────────────────────────────────────────────────────────────┘

[USER: Clicks on node]
         │
         ▼
┌─────────────────────────┐
│ Node click event        │
└─────────────────────────┘
         │
         ▼
┌──────────────────────────────────────────────────────────────────────┐
│ setupNodeInteractions → click handler                                │
└──────────────────────────────────────────────────────────────────────┘
         │
         ├─> Hide all resize handles on other nodes:
         │   nodeLayer.querySelectorAll('.handles-group').forEach(
         │     group => group.style.display = 'none'
```

```
          │   )
          │
          ├─> Show this node's resize handles:
          │   const handlesGroup = nodeGroup.querySelector('.handles-
group')
          │   handlesGroup.style.display = 'block'
          │
          └─> Emit 'node:selected' event
          │
          ▼
✓ 8 resize handles appear (one at each corner and side)

═══════════════════════════════════════════════════════════════════

[USER: Clicks on a resize handle (e.g., southeast corner)]
          │
          ▼
┌──────────────────────────────────────────────────────────────────┐
│ Handle mousedown → handleResizeStart(e, nodeId, handle)          │
└──────────────────────────────────────────────────────────────────┘
          │
          ├─> e.stopPropagation()  (Don't trigger node drag/select)
          │
          ├─> Log: "Starting resize of node {nodeId} from handle
{handle.id}"
          │
          └─> Emit event:
              │
              eventBus.emit('node:resizestart', {
                nodeId: nodeId,        ← ✓ Now correct! (was undefined)
                handleId: handle.id,   ← 'nw', 'n', 'ne', 'e', 'se', 's',
'sw', 'w'
                clientX: e.clientX,
                clientY: e.clientY
              })
          │
          ▼
┌──────────────────────────────────────────────────────────────────┐
│ [app.js] Receives 'node:resizestart' event                       │
└──────────────────────────────────────────────────────────────────┘
          │
          ├─> this.stateManager.setViewportMode("resizing")
          │
          └─> Store resize state:
              this.resizing = {
                nodeId: data.nodeId,
                handleId: data.handleId,
                startX: data.clientX,
                startY: data.clientY
              }
          │
          ▼
┌──────────────────────────────────────────────────────────────────┐
│ [NodeController] Can handle resize logic                         │
```

```
└──────────────────────────────────────────────────────────────────────┘
          │
          └─> Update node width/height based on mouse movement
          │
          ▼
     [USER: Drags handle]
          │
          ▼
     Node resizes in real-time!

┌──────────────────────────────────────────────────────────────────────┐
│                    RESIZE HANDLES LAYOUT                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│       nw ──────────── n ──────────── ne                              │
│        □              □              □                               │
│        │                             │                               │
│        │                             │                               │
│       w □          NODE             □ e                             │
│        │                             │                               │
│        │                             │                               │
│        □              □              □                               │
│       sw ──────────── s ──────────── se                              │
│                                                                      │
│   Each handle:                                                       │
│     - 8x8 px square                                                  │
│     - White fill, blue stroke                                        │
│     - Specific cursor (nw-resize, n-resize, etc.)                    │
│     - data-handle-id attribute                                       │
│     - data-node-id attribute ← Added!                                │
│     - mousedown → handleResizeStart()                                │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

6. Event System Flow

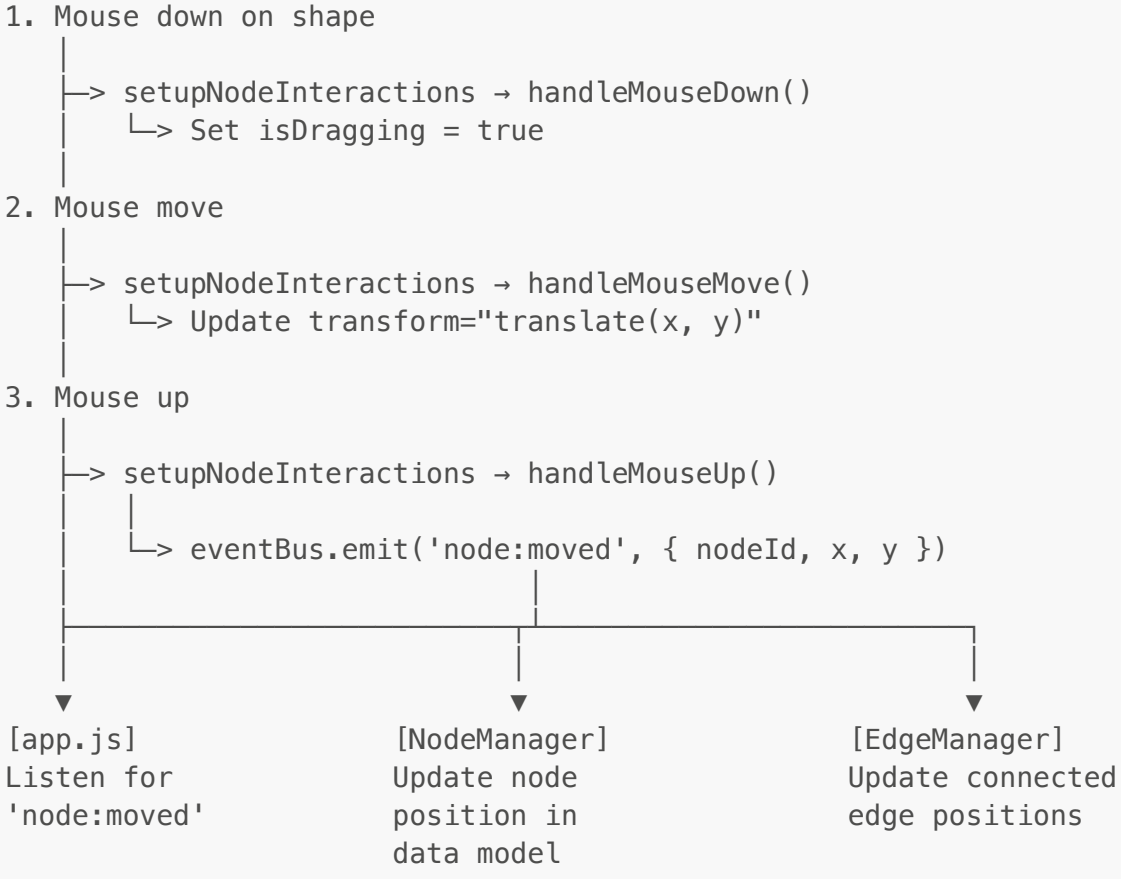
```
┌──────────────────────────────────────────────────────────────────────┐
│                    EDITOR EVENT FLOW                                 │
└──────────────────────────────────────────────────────────────────────┘


┌──────────────┐
│ Editor.js    │ Emits events
└──────┬───────┘
       │
       ├──> 'node:moved'         { nodeId, x, y }
       ├──> 'node:selected'      { nodeId, nodeData }
       ├──> 'node:edit'          { nodeId, nodeData }
       ├──> 'port:mousedown'     { nodeId, portId, portPosition, clientX,
clientY }
       ├──> 'node:resizestart'   { nodeId, handleId, clientX, clientY }
```



═══

DETAILED EVENT FLOW

[Example: User drags a node]



═══

EVENT LISTENING PATTERN

```
// In app.js:
this.eventBus.on('node:moved', (data) => {
  // Update NodeManager
  this.nodeManager.updateNodePosition(data.nodeId, data.x, data.y);

  // Update connected edges
  this.edgeManager.updateNodeEdges(data.nodeId);
});

// In StatusBar:
this.eventBus.on('node:selected', (data) => {
  // Show node info in status bar
  this.updateDisplay(`Selected: ${data.nodeId}`);
});

// In RightInspector:
this.eventBus.on('node:selected', (data) => {
  // Show node properties
  this.displayProperties(data.nodeData);
});
```

---

7. SVG Layer Structure

```
┌─────────────────────────────────────────────────────────────────────┐
│                    SVG LAYER HIERARCHY                              │
└─────────────────────────────────────────────────────────────────────┘

<svg class="editor-canvas" width="100%" height="100%">
  │
  ├─> <defs>                                    [Z-index: N/A]
  │   │                                         Reusable definitions
  │   ├─> <marker id="arrowhead">              Arrow for edges
  │   ├─> <pattern id="grid">                  Grid pattern
  │   └─> <linearGradient>                     Gradients, etc.
  │
  └─> <g id="viewport-group">                   [Zoom/Pan transform]
      │
      ├─> <g id="grid-layer">                   [Z-index: 1]
      │   │                                     Background grid
      │   └─> <path d="..."> (grid lines)
      │
      ├─> <g id="edge-layer">                   [Z-index: 5]
      │   │                                     Connections
      │   ├─> <g data-edge-id="edge_1">
      │   │   ├─> <path> (connection line)
      │   │   └─> <text> (edge label)
      │   │
      │   └─> <g data-edge-id="edge_2">
      │       └─> ...
      │
```

```
        ├─> <g id="node-layer">                    [Z-index: 10]
        │   │                                      Shapes
        │   ├─> <g data-node-id="node_1">
        │   │   ├─> <rect class="shape-element"> ← Main shape
        │   │   ├─> <text class="node-label">    ← Label
        │   │   ├─> <g class="ports-group">      ← Ports (hidden)
        │   │   │   ├─> <circle> (top)
        │   │   │   ├─> <circle> (right)
        │   │   │   ├─> <circle> (bottom)
        │   │   │   └─> <circle> (left)
        │   │   └─> <g class="handles-group">    ← Resize (hidden)
        │   │       ├─> <rect> (nw)
        │   │       ├─> <rect> (n)
        │   │       ├─> ... (8 handles)
        │   │       └─> <rect> (w)
        │   │
        │   └─> <g data-node-id="node_2">
        │       └─> ... (same structure)
        │
        └─> <g id="overlay-layer">                 [Z-index: 50]
            │                                      Selection, guides
            ├─> <rect class="selection-box">
            └─> <line class="guide">

═══════════════════════════════════════════════════════════════════

                    LAYER VISIBILITY RULES

Grid Layer:
  ✓ Always visible (if grid.enabled = true)
  ✓ Not affected by node operations

Edge Layer:
  ✓ Below nodes (so nodes appear on top)
  ✓ Updated when nodes move
  ✓ Clickable for selection

Node Layer:
  ✓ Main interactive layer
  ✓ Contains all node elements
  ✓ Ports shown on hover
  ✓ Resize handles shown on selection

Overlay Layer:
  ✓ Above everything
  ✓ Selection boxes, alignment guides
  ✓ Temporary visual feedback
```

---

8. Complete User Interaction Flow

```
┌──────────────────────────────────────────────────────────────────────┐
│              COMPLETE USER INTERACTION MAP                           │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ USER ACTION                        EDITOR RESPONSE                   │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│ 1. Drop shape from palette                                           │
│    └─> handleDrop()                                                  │
│        └─> Emit 'shape:dropped' → app.js creates node                │
│                                                                      │
│ 2. Click on node                                                     │
│    └─> setupNodeInteractions → click handler                         │
│        ├─> Hide other resize handles                                 │
│        ├─> Show this node's resize handles                           │
│        └─> Emit 'node:selected'                                      │
│                                                                      │
│ 3. Drag node                                                         │
│    └─> setupNodeInteractions → drag handlers                         │
│        ├─> mousedown: Start drag, record position                    │
│        ├─> mousemove: Update transform continuously                  │
│        └─> mouseup: Emit 'node:moved' with final position            │
│                                                                      │
│ 4. Hover over node                                                   │
│    └─> setupNodeInteractions → mouseenter                            │
│        └─> Show ports (4 blue circles)                               │
│                                                                      │
│ 5. Mouse leaves node                                                 │
│    └─> setupNodeInteractions → mouseleave                            │
│        └─> Hide ports                                                │
│                                                                      │
│ 6. Click on port                                                     │
│    └─> handlePortMouseDown()                                         │
│        └─> Emit 'port:mousedown' with nodeId, portId                 │
│                                                                      │
│ 7. Click on resize handle                                            │
│    └─> handleResizeStart()                                           │
│        └─> Emit 'node:resizestart' with nodeId, handleId             │
│                                                                      │
│ 8. Double-click on node                                              │
│    └─> setupNodeInteractions → dblclick                              │
│        └─> Emit 'node:edit'                                          │
│                                                                      │
│ 9. Click on canvas (not on node)                                     │
│    └─> handleClick()                                                 │
│        ├─> Hide all resize handles                                   │
│        └─> Emit 'canvas:clicked'                                     │
│                                                                      │
│ 10. Mouse wheel on canvas                                            │
│     └─> handleWheel()                                                │
│         ├─> Calculate new zoom                                       │
│         └─> updateTransform()                                        │
```

```
│                                                                    │
│ 11. Right-click on canvas                                          │
│     └─> handleContextMenu()                                        │
│         └─> Emit 'canvas:contextmenu'                              │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Summary

Key Components:

1. **SVG Structure** - Layered architecture (grid → edges → nodes → overlay)
2. **Node Rendering** - Multi-step process creating shape, label, ports, handles
3. **Drag System** - Three-phase (mousedown → mousemove → mouseup)
4. **Event System** - EventBus distributes events to all listeners
5. **Interaction Handlers** - setupNodeInteractions manages all node interactions

Critical Data Flows:

```
nodeId → Always passed correctly to all handlers ✓
Ports → Have data-node-id attribute ✓
Handles → Have data-node-id attribute ✓
Events → Include nodeId in all emissions ✓
```

State Management:

```
Drag State:   isDragging, dragStartX/Y, nodeStartX/Y
Viewport:     x, y, zoom
Grid:         enabled, size, visible
Layers:       gridLayer, edgeLayer, nodeLayer, overlayLayer
```

This architecture ensures clean separation of concerns and makes the editor extensible for future features!